Logical Memory, Physical State, and Search Cost: A Controlled Scaling Study of Progressive Retrieval Attention

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 16, 2026

Abstract

Long-context claims often collapse three quantities: text a system can address, key/value (K/V) state present in attention, and work needed to find useful state. Progressive Retrieval Attention (PRA) separates them. Compact gists index logical references; a router selects evidence; only selected layer-native K/V enters the transformer's attention operation. We ask whether logical memory can grow while physical attention state follows the task working set rather than the full address space. This paper reports a controlled first gate, not a language-model scaling law. Across 5 seeds, a fixed four-region retrieval task grows from 32,768 to 8,388,608 logical tokens. At a preregistered 128-token active budget, exact evidence recall remains 1.000–1.000, yielding a 65,536:1 logical-to-active ratio at the largest point. Exact GPU search latency grows 30.19×. A four-probe coarse index makes 102.4× fewer comparisons and is 2.85× faster at 8M tokens while preserving the planted evidence, although it overlaps only 0.391 of the exact top-16 ranking. When evidence regions and chain depth grow, required active K/V grows with evidence count. The result supports bounded *selected attention state* in a separable synthetic geometry. It does not establish bounded total memory, end-to-end language-model quality, production serving efficiency, or "infinite context."

1 Introduction

Transformer context is usually discussed as one number. Operationally, at least three numbers matter. A system may address a large body of source text, search a smaller index over that body, and expose only a small set of K/V states to a decoder. Dense long context makes the three nearly coincide. Retrieval-augmented generation separates source storage from the prompt but re-encodes selected text [7, 5]. KV compression reduces resident state after the model has processed a sequence [18, 8, 10]. PRA instead treats compact gists as addresses and preserves selected token detail in the transformer's native K/V representation [15, 16].

This decomposition changes the scaling question. We do not ask whether a model can attend densely to an unbounded sequence. We ask whether externally addressable *logical memory* can grow by orders of magnitude while active native-K/V and layer-token attention state remain tied to the evidence working set. Search work, index bytes, transfer, and model quality remain separate costs. The distinction echoes working-set analysis in virtual memory: address space and resident state are related but not identical [2].

The first experiment is intentionally controlled. It plants a fixed four-node working set inside successively larger pools of distractor chunks and measures exact and coarse-to-fine native-key retrieval on one CUDA system. A second sweep increases evidence regions and chain depth. This design isolates address-space scaling from task-difficulty scaling, but it does not exercise a pretrained language model. Paper 4's adaptation ladder is included only as inherited plasticity calibration, never as a point on the routing curve [13].

Contributions. We provide (i) an explicit scaling vocabulary and accounting model for logical memory, active K/V, layer sparsity, and search; (ii) a five-seed controlled sweep from 32K to 8M logical tokens and from 32 to 1024 active K/V tokens; (iii) exact, indexed, adaptive-effort, dispersion, hardware, and routing-concurrency measurements with raw CSVs and fit residuals; and (iv) a claim audit that marks every unmeasured model, serving, hardware, and cost cell.

2 Scaling Variables and Hypotheses

Let L denote logical reference tokens and $N = L/G_s$ the number of searchable memory nodes at search granularity G_s. Let B be the number of native-K/V tokens materialized at one PRA consumer layer. If $\mathcal{P}$ is the set of consumer layers, total physical attention state is

$$C_{\mathrm{KV}} = \sum_{\ell \in \mathcal{P}} N_{\mathrm{KV}}(\ell). \tag{1}$$

With a shared budget B at $P = |\mathcal{P}|$ layers, $C_{\mathrm{KV}} = PB$. This layer-token measure prevents a sparse two-layer integration from being confused with the same token count injected at every layer.

PRA uses three granularities:

$$G_{\mathrm{encode}} \neq G_{\mathrm{search}} \neq G_{\mathrm{materialize}}. \tag{2}$$

Encoding blocks keep the base model within its safe context. Search chunks define addressable nodes and gists. Materialized spans define physical native-K/V. A coarse search configuration additionally has query facets F, roots R, neighbors K, hops H, probes, thresholds, and retries. These variables should be reported, not hidden behind "context length."

Our primary hypothesis is deliberately conditional:

$$B = O(W) \quad \text{with respect to } L, \tag{3}$$

where W is task working-set complexity, such as evidence regions and chain depth. It predicts bounded selected attention state at fixed difficulty, not bounded index bytes or search latency. Secondary hypotheses ask whether indexed search grows more slowly than exact search, whether sparse consumer layers multiply physical savings, and whether learning changes the amount of useful external memory. Each remains empirical.

3 Mechanism and Cost Boundaries

PRA's runtime pipeline is

$$\text{logical sources} \to \text{model-safe encoding} \to \text{native K/V} \to \text{search chunks} \to \text{gists/index} \to \text{routing} \to \text{budgeted materialization} \to \text{direct+memory attention}.$$

The gist is an address summary, not a replacement value. Once a node is selected, the corresponding native keys and values join local K/V under the decoder's ordinary attention softmax. Papers 2.5 and 3 study discovery and physical disclosure separately [12, 14]; Paper 3.5 adds retry and serving control [11].

This pipeline creates four memory pools. Source/reference bytes may remain on CPU or external storage. The gist index occupies CPU or GPU memory. Selected detail K/V becomes GPU-resident attention state. Local decode K/V remains the ordinary model cache. Only the third pool is counted as active PRA K/V here. Moving an index off GPU may reduce HBM while increasing transfer or search latency; it does not make the index disappear.

For hidden width d, bytes per selected token per consumer layer are approximately $2d\,b$, where b is bytes per K or V scalar. Thus

$$M_{\mathrm{active}} \approx 2dbC_{\mathrm{KV}}. \tag{4}$$

Dense context instead retains K/V across its participating layers. This accounting is exact only after topology, grouped-query attention, dtype, paging, and fragmentation are specified.

4 Experimental Protocol

4.1 Matched fixed-difficulty task

The primary dataset contains eight queries per seed. Each query has four evidence nodes planted in a clustered 64-dimensional key space. Evidence identity and content are held fixed while distractor chunks grow. Logical memory is 32K, 128K, 512K, 2M, and 8M tokens. Search chunks contain 32 logical tokens; selected nodes disclose eight physical K/V tokens. Active budgets are 32, 64, 128, 256, 512, and 1024 tokens. The reported layer-token count assumes two PRA consumers among eight layers. Five fixed seeds are used.

The query and its evidence share a latent direction; distractors occupy clustered background directions. This is a separable diagnostic geometry. It asks whether the implementation preserves a fixed working set as the address pool grows. It does not model ambiguous language, compositional answer generation, or learned query/key drift.

4.2 Search and adaptive effort

Exact search computes one query-key matrix product followed by `torch.topk`. Coarse-to-fine search scores deterministic cluster centroids, probes 1, 4, or 16 clusters, and ranks only their members. Both execute on the same PyTorch/CUDA stack. Timings use a warm-up, seven synchronized repeats, and p50/p95 reporting. The prototype uses Python loops per query; it is not a fused ANN kernel.

An oracle-free adaptive controller starts at 32 active tokens and checks the score gap at its current boundary. Thresholds are 0, 0.03, 0.06, and 0.10. It escalates through the same budget ladder when confidence is insufficient. Evidence labels are consulted only after selection to score recall.

4.3 Difficulty, hardware, and inherited calibration

Difficulty scaling crosses 1, 2, 4, or 8 evidence regions with chain depth 1, 2, or 4 at fixed 512K logical tokens. We record the first budget reaching 90% evidence recall. Routing-only concurrency uses batches of 1, 4, and 8 queries. CPU and CUDA microbenchmarks are reported separately; Apple Silicon is an empty preregistered cell.

Every model result is labeled Frozen, LoRA, Full-weight continued, or PRA-native. The only measured model ladder in this branch is inherited from Paper 4's 445,824-parameter controlled model. Gemma 3 270M, approximately 1B, and approximately 4B are planned cells. They are not interpolated or plotted as quality observations.

5 Logical-Memory Scaling

Table 1 reports the primary operating point. Exact evidence and complete-path recall are 1.0 at every memory size and every seed. Active K/V is fixed at 128 tokens, so the logical-to-active ratio rises from 256:1 to 65,536:1. This is the intended bounded-selected-state result. It is also partly structural: the budget was fixed by protocol, and the planted geometry remains separable. The empirical content is that recall did not collapse as 261,120 additional search nodes were introduced.

Table 1: Fixed-difficulty scaling, averaged over five seeds. Indexed latency uses four probes. "Ratio" is logical tokens divided by active native-K/V tokens. Index MiB counts the float32 key matrix and centroids resident on the measured device; it is not active attention state.

Logical tokens	Nodes	Recall	Active K/V	Ratio	Exact ms	Indexed ms	Index MiB
32,768	1,024	1.000	128	256	0.195	2.089	0.26
131,072	4,096	1.000	128	1,024	0.277	2.273	1.02
524,288	16,384	1.000	128	4,096	0.620	2.101	4.03
2,097,152	65,536	1.000	128	16,384	1.691	2.146	16.06
8,388,608	262,144	1.000	128	65,536	5.887	2.063	64.12

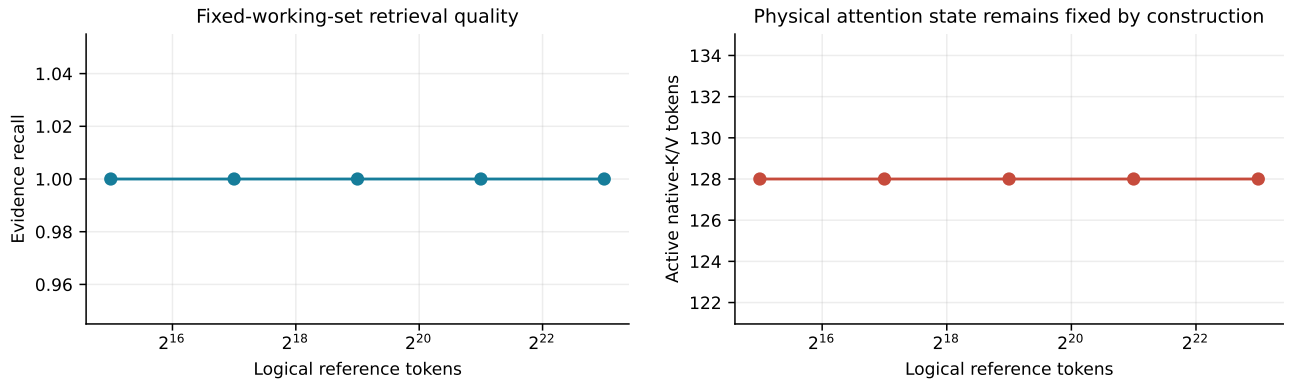

Figure 1: Address-space scaling at fixed task difficulty. Recall remains saturated while selected attention state is constant. Confidence intervals collapse because all five seeds agree; the result should be read with the task-separability caveat.

The routing index does grow. Its measured float32 footprint rises from 0.26 MiB to 64.1 MiB. Peak allocated HBM grows further because temporary key-generation and scoring tensors are included. PRA therefore changes the relationship between logical memory and *attention state*; this pilot does not show constant total HBM.

6 Active K/V and Working-Set Scaling

At fixed four-node difficulty, the smallest 32-token budget already retrieves all evidence. Larger budgets add no quality. This clean knee is diagnostic but not a general operating recommendation: natural data may require surrounding context, multiple chunks per source, or less precise routes.

The dispersion experiment is more informative. Required K/V stays at the minimum 32 tokens for one to four evidence nodes, then rises to 64, 128, and 256 tokens for 8, 16, and 32 evidence nodes. Above the minimum allocation quantum, physical state is eight tokens per planted evidence node. Region count and chain depth with the same product collapse to the same requirement in this generator; it does not yet expose a separate depth penalty.

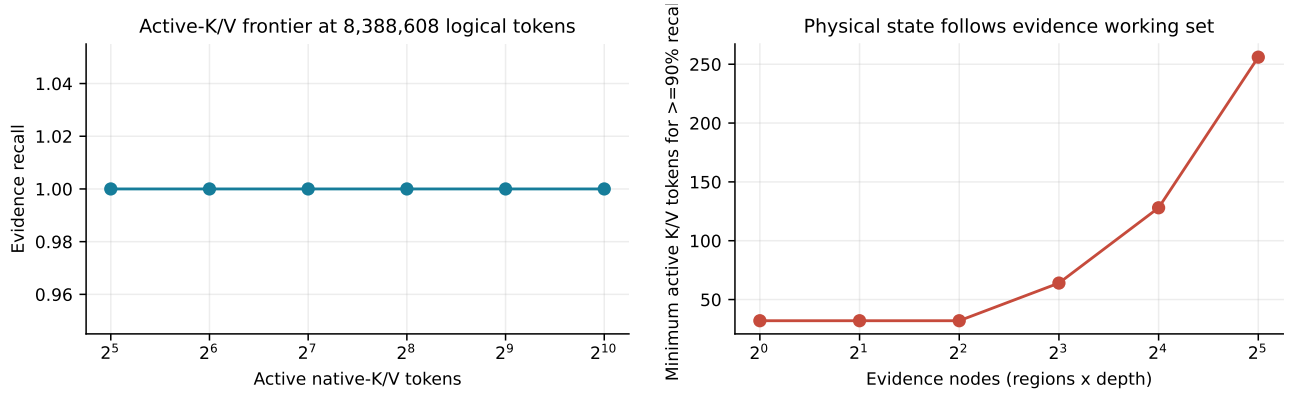

Figure 2: Left: fixed-task active-K/V sweep at 8M logical tokens. Right: first budget reaching 90% recall as the evidence working set grows. The second curve, not the saturated first, tests the working-set hypothesis.

Table 2: Minimum active K/V reaching 90% evidence recall. "Cells" counts seed-region-depth combinations sharing an evidence-node count.

Evidence nodes	Required K/V	95% CI	Cells
1	32.0	0.0	5
2	32.0	0.0	10
4	32.0	0.0	15
8	64.0	0.0	15
16	128.0	0.0	10
32	256.0	0.0	5

Layer sparsity multiplies this result. At the primary point, 128 tokens at two consumers give 256 layer-token states. Injecting the same state into all eight layers would give 1,024. Whether two consumers preserve language-model quality is not answered by retrieval recall and must be tested in the matched model ladder.

7 Search and Index Scaling

Exact GPU search grows from roughly 0.2 ms to roughly 6 ms over the 256-fold logical-memory increase, a 30.19× rise. A power fit gives an exponent near 0.6, but the lowest-AIC saturating form places its scale at the boundary of the search grid and is nearly linear over the observed range. Five hardware-specific points cannot identify an asymptotic law. We therefore report raw points and residuals rather than calling the curve a power law.

The coarse index behaves differently. Four probes examine cluster centroids plus candidate members. At 8M logical tokens it performs 102.4× fewer dot products than exact search and is 2.85× faster. At smaller pools,

Python dispatch dominates and indexed search is slower. This crossover is a measured failure regime for simplistic "ANN is faster" claims.

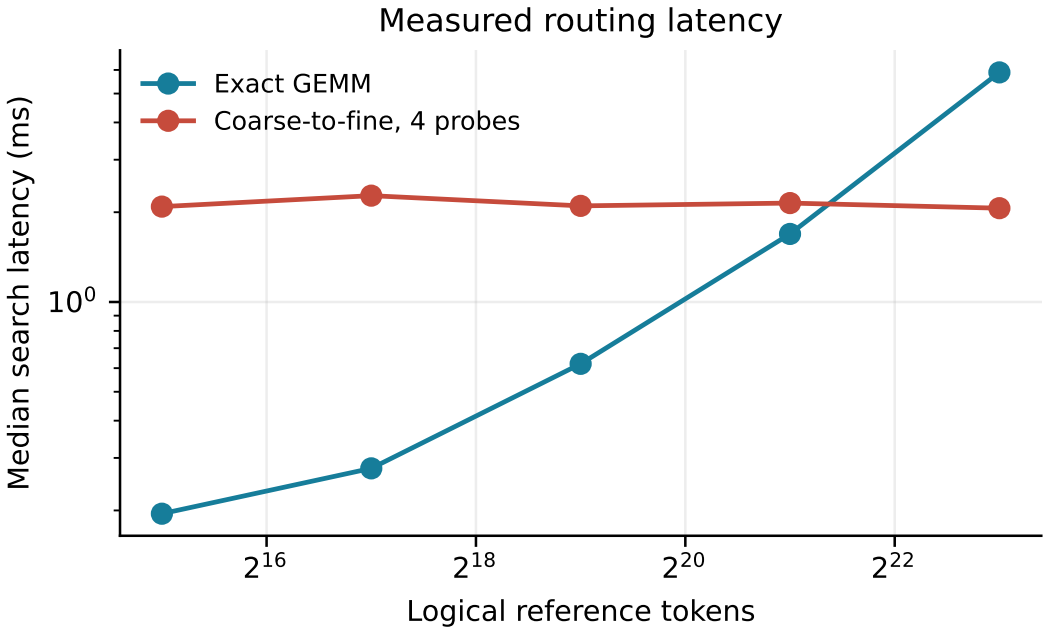

Figure 3: Exact and four-probe indexed routing latency. The indexed prototype has an approximately 2.2 ms software floor but crosses exact GEMM at the largest pool. These are routing-only timings on one GPU.

Task recall and nearest-neighbor equivalence are different. The indexed route recovers every planted evidence node at every scale, but its exact top-16 overlap falls to 0.391 at 8M. The omitted exact neighbors are distractors under this diagnostic. A natural-language router cannot assume that such disagreements are harmless. Future ANN evaluation must report both evidence recall and overlap with exact search, plus index build/update time and p95 tails.

8 Adaptive Effort

The score-gap controller never escalates in this task. It uses 32 active tokens, recovers all four evidence nodes, and has zero escalation rate for every threshold and memory size. This is consistent with bounded expected effort, but it is not evidence that adaptive effort remains bounded under ambiguity. Rather, it shows that the controller recognizes the intentionally wide planted margin.

Useful adaptive scaling requires a calibrated mixture of easy, ambiguous, and failed routes. Difficulty should grow through closer distractors, missing roots, multi-hop discovery, and competing answer paths. Expected effort can then be conditioned on difficulty rather than averaged into an easy-task constant.

9 Parameters, Plasticity, and External Memory

Neural scaling laws relate loss to model, data, and compute under carefully matched training regimes [4, 3]. External memory adds at least three axes: logical memory L, active K/V B, and search effort S. The desired surface is

$$Q = f(N_\theta, L, B, S, C_{\text{train}}), \tag{5}$$

not a claim that a small PRA model automatically matches a larger dense model.

Paper 4 provides one plasticity slice at 445,824 base parameters. Frozen PRA reaches roughly 0.30 evidence-only accuracy; Consumer and Interface LoRA reach roughly 0.82 and 0.88; Broad LoRA, full continued training, and PRA-native training approach 0.95–0.99 on the controlled task. Those observations establish that learning changes

memory usability. They do not estimate a parameter exponent because model size is fixed, task structure differs from this routing benchmark, and memory identity is oracle-selected.

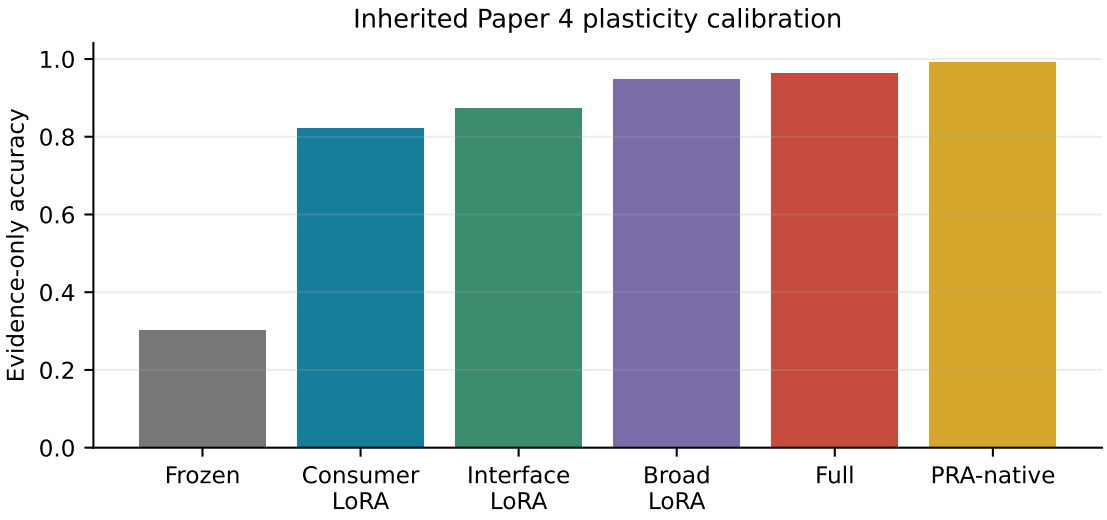

Figure 4: Inherited Paper 4 calibration. Regimes share a controlled oracle-memory task but do not form a model-size scaling curve.

Table 3: Matched model ladder status. No quality value is inferred for pending cells. The 4B run is sponsorship-gated.

Checkpoint	Parameters	Status
Paper 4 controlled PRA	445,824	measured calibration
Gemma 3 270M	270M nominal	pending matched task
Gemma 3 1B	999,885,955	pending matched task
Gemma 3 4B	about 4B	sponsorship gate

The next experiment must first compare PRA and native attention at the same model size, then compare PRA(N) with Native(cN). It must report ordinary language modeling, reasoning, memory-heavy QA, and long-context retrieval separately. Frozen, LoRA, full-weight continued, and PRA-native checkpoints require distinct curves because adaptation capacity itself changed the controlled outcome.

10 Routing Throughput and Hardware

Batched exact routing amortizes launch and matrix costs. At 8M logical tokens, the measured routing-only throughput rises from about 287 queries/s at batch one to about 1,350 queries/s at batch eight, while per-batch latency grows from about 3.5 to 5.9 ms. These are not end-to-end requests: tokenization, reference resolution, H2D transfer, K/V gather, model prefill, decoding, networking, and scheduler contention are absent.

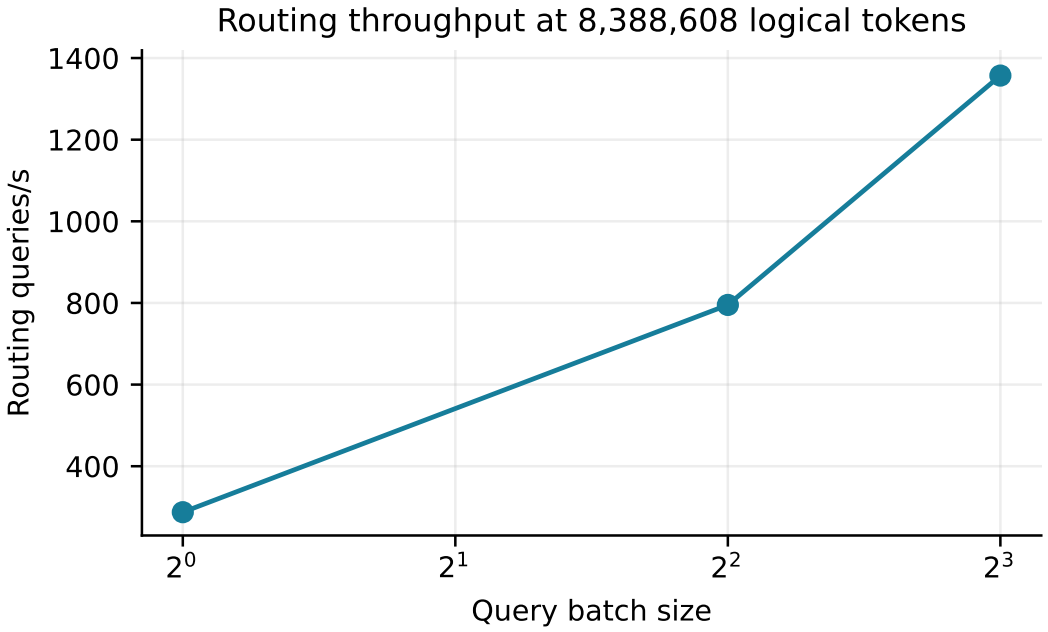

Figure 5: Routing-only exact-search throughput at 8M logical tokens. Batching improves query throughput but does not establish serving throughput or TPOT.

The hardware slice covers an Intel mobile CPU and an NVIDIA GeForce GTX 950M. Single-seed device timings vary enough that we make no normalized CPU-versus-GPU claim. Apple Silicon, a modern CUDA accelerator, and a production serving stack remain required. TTFT, TPOT, GPU utilization, throughput per dollar, and cost per million tokens are intentionally blank in the artifacts rather than estimated from unrelated hardware.

11 Baselines and Pareto Frontiers

Dense native context, text RAG, iterative RAG, fixed PRA, adaptive PRA, and KV compression solve overlapping but different state-management problems. Dense attention preserves token-level interactions but pays context-dependent K/V and attention cost. RAG searches external text and usually re-encodes selected passages [7, 1]. Memorizing Transformers and kNN-LMs retrieve stored representations [17, 6]. H2O, SnapKV, ClusterKV, and KIVI reduce or organize K/V after context processing [18, 8, 9, 10]. PRA's distinctive claim is addressable reference identity followed by sparse native-K/V disclosure.

The supplied baseline CSV includes only analytical state accounting: dense native context uses L active tokens, a top-four text RAG baseline imports four 32-token chunks, and the primary PRA point imports sixteen eight-token spans. No quality or latency is attached. Mixing these rows into a measured Pareto curve would be false precision. The current measured frontier is routing-only and mostly saturated in quality; a meaningful system frontier awaits matched output quality, TTFT, TPOT, HBM, and cost.

12 Candidate Laws and What the Fits Mean

We compare constant, linear, logarithmic, power, and saturating-exponential forms. All raw points, predictions, residuals, RMSE, R^2, and AIC values are emitted. Constant fits correctly describe selected K/V and evidence recall in the fixed task. Exact-search timing is fit well by linear and boundary-scale saturating forms; their small AIC difference does not establish saturation. Indexed timing is dominated by a near-constant software floor across this range.

These fits are descriptive. Five geometrically spaced points on one device do not justify extrapolation beyond 8M, and a fully resident float32 index is not the architecture expected at billion-node scale. Piecewise behavior is plausible: kernel launch dominates small pools, memory bandwidth dominates larger exact search, and index residency or cache misses may create later cliffs.

The observed evidence requirement is closer to

$$B \approx 8W \tag{6}$$

above a 32-token minimum, where W is the number of evidence nodes. This is a property of the generator's eight-token disclosure policy, not a universal PRA constant. Natural tasks must estimate W empirically.

13 Failure Regimes and Extrapolation Limits

The pilot exposes several limits.

1. **Task saturation.** Every fixed-task query succeeds at the smallest budget, so quality headroom is absent and adaptive retry is untested.
2. **Index growth.** Active attention state is fixed, but the resident float32 routing index reaches 64.1 MiB at 8M logical tokens.
3. **Ranking divergence.** Indexed evidence recall is perfect while exact top-16 overlap falls to 0.391.
4. **Small-pool overhead.** Python coarse routing loses to one GPU GEMM until the largest pool.
5. **No output model.** Retrieval success does not prove NLL, EM/F1, or generation improvement.
6. **No production path.** Paging, transfer, cache thrashing, concurrent materialization, and decode interference are absent.

Further experiments should actively search for retrieval collapse, ANN recall loss, controller over-escalation, active-budget growth, latency cliffs, and model-size reversal. A scaling failure is a result, not a nuisance point to omit.

14 The 4B Sponsorship Gate

The 270M and 1B ladder points should calibrate any 4B prediction. Before a 4B run, the experiment package must contain: matched tasks and checkpoint regimes; an expected information gain over the 1B posterior; accelerator-hour and token budgets; a stop-loss rule; artifact retention; and explicit success/failure predictions for output quality, active K/V, and latency. The present pilot does not satisfy this gate because neither smaller Gemma point has been measured on the scaling grid.

The highest-value next sequence is: (1) make the fixed task less separable and calibrate adaptive escalation; (2) run Gemma 3 270M Frozen and LoRA at 32K–512K; (3) repeat the retained settings at 1B; (4) add QASPER and controlled multi-hop data; (5) integrate paged CPU detail K/V and a production ANN; and only then decide whether 4B resolves an uncertainty that smaller runs cannot.

15 Conclusion

PRA changes the context accounting problem by separating addressable logical memory from selected physical attention state. In this controlled five-seed study, logical memory grows 256-fold to 8M tokens while 128 active native-K/V tokens preserve perfect planted-evidence recall. Physical state instead grows with evidence working-set size. Search and index costs do not vanish: exact latency rises, the resident index grows, and approximate ranking diverges even when task recall survives.

This is a useful first gate because it makes the central hypothesis falsifiable and its boundaries visible. It is not yet a language-model scaling result. The next evidence must connect the same accounting to matched pretrained models, natural tasks, production memory placement, and end-to-end latency and cost.

A Artifact Contract and Reproduction

The experiment entry points are in `experiments/paper5_scaling_laws/`: the runner is `run_scaling_study.py` and the summarizer is `summarize_scaling_study.py`. The configuration manifest records seeds, device, PyTorch and Python versions, git revision, granularities, budgets, probes, and thresholds. Raw files include model, logical-memory, active-K/V, search-index, adaptive-effort, dispersion, parameter-memory, hardware, serving, baseline, and training-compute tables. Fit JSON and CSV preserve alternative families and residuals. `scaling_claim_audit.md` is the normative claim boundary.

The complete run is:

```
python -m experiments.paper5_scaling_laws.run_scaling_study
python -m experiments.paper5_scaling_laws.summarize_scaling_study
cd docs/papers/paper5_pra_scaling_laws
latexmk -pdf -interaction=nonstopmode paper.tex
```

The routing benchmark allocates only gist/key surrogates, not 8M token-detail K/V. CPU reference bytes and dense baseline state are analytical accounting fields. GPU index bytes, peak allocation, search latency, recall, comparisons, and routing throughput are measured. Empty strings in systems CSVs mean unavailable, not zero.

B Preregistered Headline-Plot Status

Fourteen plot targets are generated. Quality/logical memory, active K/V/logical memory, search latency, resident HBM, quality/active K/V, quality/layer-token K/V, adaptive effort, routing concurrency, plasticity calibration, routing Pareto, and dispersion are measured or inherited as labeled. Parameter-by-memory coverage is a status matrix because only the toy point exists. Cost-quality is an explicit "not measured" panel. Native/RAG/KV quality Pareto remains absent until matched quality and latency exist.

References

[1] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.

[2] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[4] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[5] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

[6] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[8] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: `https://arxiv.org/abs/2404.14469`.

[9] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.

[10] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, pages 32332–32344, 2024.

[11] Jorge Simao. Adaptive progressive retrieval attention: Routing, retry, and serving control. Companion Paper 3.5 manuscript, 2026.

[12] Jorge Simao. Iterative progressive retrieval attention: Bounded associative discovery over latent gist memory. Companion Paper 2.5 manuscript, 2026.

[13] Jorge Simao. Learning to use sparse native-KV memory: A controlled adaptation ladder for progressive retrieval attention. Companion Paper 4 manuscript, 2026.

[14] Jorge Simao. Oracle-first native-KV materialization for progressive retrieval attention. Companion Paper 3 manuscript, 2026.

[15] Jorge Simao. Progressive retrieval attention: Bounded sparse native-KV for addressable logical memory. Companion Paper 1 manuscript, 2026.

[16] Jorge Simao. Progressive retrieval attention for pretrained transformers: Sparse native-K/V memory with semantic routing. Companion Paper 2 manuscript, 2026.

[17] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[18] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: `https://openreview.net/forum?id=RkRrPp7GKO`.